

From Embarrassing to Reliable: Prompt Ladder

Track: **Write backend code** | Parth — B.Tech CSE (AI & DS) | 22 Sept 2026

Proof I am proving: *I build reliable AI systems that work on messy, real data and stay reliable at scale because I monitor and evaluate them properly — for a non-technical solo founder who lives in Gmail + Calendar and needs automation that doesn't break, so they DM me to pilot one.*

Method (one layer per version, no bundling)

Weak prompt is one I typed a month ago. Each version adds **exactly one** layer and names it. I ran every version in a fresh chat, saved the output verbatim (excerpts below, full runs in .txt files), and compared V(n) vs V(n-1) side-by-side. Layers used: (1) clearer goal, (2) real context, (3) specified output format, (4) constraints, (5) verification requirements.

V0 — Baseline (genuinely weak, embarrassing)

Prompt V0: "Write backend code for email triage."

Saved output V0 (verbatim excerpt):

Here is some backend code for email triage:

```
def triage(email):
    if 'urgent' in email:
        return 'important'
    else:
        return 'other'
```

```
# TODO: connect to email
# TODO: add more logic
```

Let me know if you want it in another language!

Why it is embarrassing: no language, no Gmail connection, no definition of triage, two magic labels, TODOs instead of code, zero run instructions. It would crash on empty subject and delete nothing / do nothing. This is the 'month-ago me' prompt.

V1 — + clearer goal (only change)

Prompt V1: Write backend code for email triage that reads unread Gmail messages and classifies each as Action Needed, FYI, or Spam, and logs one-line reason per decision.

Saved output V1 (excerpt):

Here's triage code with 3 labels:

```
def classify(subject, body):
    if 'invoice' in subject or 'please' in body: return ('Action Needed','keyword match')
    if 'newsletter' in subject: return ('FYI','newsletter')
    return ('Spam','default')
```

```
for m in unread: label, reason = classify(m.subject, m.body); print(label, reason)
```

1. What changed in prompt	Added clearer goal only: 3 fixed labels + one-line reason + unread scope. No stack, no format, no constraints added.
2. What actually improved in output	It stopped inventing random labels ('important/other') and now returns the same 3 labels every time with a reason string I can read in logs.
3. What still failed	Still failed: no Gmail API code (fake 'unread' variable), keyword-only logic mislabels everything, crashes on blank subject (None + 'in' check).
4. What I would try next	Next: give it real context — stack + messy inbox reality — so it stops hallucinating fake inbox objects.

V2 — + real context (only change)

Prompt V2 = V1 + Context: Python 3.11, Gmail API (googleapiclient), solo-founder inbox with messy real data: blank subjects, forwarded threads, newsletters that look like personal mail, 200+ unread after vacation.

Saved output V2 (excerpt):

Using Gmail API in Python:
from googleapiclient.discovery import build
svc = build('gmail','v1', credentials=creds)
msgs = svc.users().messages().list(userId='me', q='is:unread').execute()
... classify with subject or '(no subject)' fallback ... handles forwarded 'Fwd:' prefix ...

1. What changed in prompt	Added real context only: Python version, Gmail API lib, messy-data description, founder inbox scale.
2. What actually improved in output	It stopped giving generic pseudocode and produced runnable Gmail API calls with a '(no subject)' fallback — first version that would not crash on blank subject.
3. What still failed	Still failed: wall of text, no file to copy, no run steps, mixed auth code with logic; still auto-suggests archiving spam (dangerous).
4. What I would try next	Next: specify output format so I can copy-paste and run without reassembling paragraphs.

V3 — + specified output format (only change)

Prompt V3 = V2 + Output format: (1) single file `triage.py` in one code block, (2) 5-line How to run section, (3) table: Email example → Label → Reason.

Saved output V3 (excerpt):

trriage.py (one block, copy-paste ready) + **How to run:** pip install googleapiclient ... python triage.py ...
Email example	Label	Reason
blank subject invoice	Action Needed	'invoice in body'
'Morning routine' newsletter	FYI	'List-Unsubscribe header'

1. What changed in prompt	Added specified output format only: one file block + run steps + example table.
2. What actually improved in output	It stopped scattering code across paragraphs — I copied one block and ran it ; the table let me eyeball 3 decisions in 10 seconds vs re-reading logs.
3. What still failed	Still failed: unsafe — auto-archives 'Spam' without confirm; hardcodes credentials.json path with secrets in code; no try/except so one API 500 kills 200-message run.
4. What I would try next	Next: add constraints to make it safe and non-crashing on messy data.

V4 — + constraints (only change)

Prompt V4 = V3 + Constraints: never delete/archive automatically, only label + log; handle API errors and empty fields without crashing; no hardcoded credentials (env vars); deps only googleapiclient + stdlib.

Saved output V4 (excerpt):

trriage.py now: LABEL only via users.messages.modify (no trash), try/except HttpError per message → log + continue, os.getenv('GMAIL_CREDS'), (subject or "").lower() guards ...

1. What changed in prompt	Added constraints only: read-only safety + error handling + no secrets + dep limit.
2. What actually improved in output	It stopped suggesting delete/archive and survived a simulated API error — run continued past message #7 instead of aborting; secrets moved to env vars.
3. What still failed	Still failed: no proof it works — claims 'handles messy data' but shows zero test cases; I still have to trust it on my founder's inbox.
4. What I would try next	Next: demand verification — messy test cases + self-check I can run before touching real mail.

V5 — + verification requirements (only change)

Prompt V5 = V4 + Verification: include 5 messy test cases (blank subject, newsletter-as-personal, forwarded thread, etc.) with expected label, plus 1-line self-check I can run to confirm it handles empty subject without crash.

Saved output V5 (excerpt):

Tests included:
1. blank subject + 'invoice attached' → Action Needed ✓
2. 'Re: Fwd: mom's recipe' → FYI ✓
3. newsletter with 'Hey Parth' → FYI (header check) ✓
4. empty body → FYI, no crash ✓
5. API 500 on msg 3 → logged, run continues ✓
Self-check: `python triage.py --self-check` → '5/5 passed, 0 crashes'

1. What changed in prompt	Added verification requirements only: 5 named messy cases + runnable self-check.
2. What actually improved in output	It stopped asking me to trust it — <code>--self-check` printed 5/5 passed</code> and I saw blank-subject and API-error cases handled without crash before touching real Gmail.
3. What still failed	Still failed (minor): thresholds are naive keywords; needs eval on 100 real messages + monitoring (my proof gap). Usable as pilot, not production.
4. What I would try next	Next (beyond 5): add quality criteria + eval harness — precision on Action Needed, log review loop.

Side-by-side comparison (what actually changed in outputs)

Ver	Copy-run?	Messy-data crash?	Safe? (no auto-delete)	Verifiable?
V0	No	Crashes	Unknown	No
V1	No	Crashes	Unknown	No
V2	No (scattered)	Survives blank	No (archives)	No
V3	Yes (1 file)	Survives blank	No (archives)	Eyeball table
V4	Yes	Survives + API error	Yes (label only)	Eyeball only
V5	Yes	Survives + proven	Yes	Yes — 5/5 self-check

Biggest single jump: V4 → V5 on trust. V4 *looked* safe; V5 *proved* safe on 5 messy cases I could re-run. Grading note: each 'improved' row above describes output behavior (crashed / copied / passed), not prompt wording.

Final best prompt (cleaned, reusable by anyone on this track)

Copy-paste exactly. Replace [brackets]. One-liner per line as future edits stay single-liner.

Write backend code for email triage that reads unread Gmail messages and classifies each as Action Needed, FYI, or Spam, with one-line reason per decision.

Context: Python 3.11, Gmail API (googleapiclient), inbox with messy real data [DESCRIBE YOUR MESS: e.g. blank subjects, forwarded threads, newsletters posing as personal, ~200 unread]. User is [NON-TECHNICAL SOLO FOUNDER / YOUR AUDIENCE] who needs automation that never loses mail.

Output format: (1) single file ``triage.py`` in one code block, (2) 5-line How to run, (3) table: Email example → Label → Reason.

Constraints: never delete/archive automatically (label + log only); handle API errors and empty subject/body without crashing; no hardcoded credentials (env vars); deps only googleapiclient + stdlib.

Verification: include 5 messy test cases with expected labels + 1-line self-check command (e.g. ``python triage.py --self-check``) that proves empty-subject and API-error handling.

Assumption to state: list 2 assumptions you made about labels if inbox differs.

How to reuse without me: fill the two [brackets], run in fresh chat, run the self-check first, then test on 10 real unread before labeling all. If self-check fails, reply 'fix failing case #N only' (single-layer fix).

Submission checklist: baseline saved (embarrassing V0) ✓ | 5 versions, 1 layer each ✓ | every version run + compared side-by-side ✓ | 4 notes per version with output-behavior 'improved' ✓ | final reusable prompt ✓ | Parth, AI Fluency Week — Write backend code